

Pollard Rho

分解质因数

引入

给定一个正整数，试快速找到它的一个[非平凡因数](#)。

考虑朴素算法，因数是成对分布的，的所有因数可以被分成两块，即 和 。只需要把 里的数遍历一遍，再根据除法就可以找出至少两个因数了。这个方法的时间复杂度为 。

当 时，这个算法的运行时间我们是无法接受的，希望有更优秀的算法。一种想法是通过随机的方法，猜测一个数是不是 的因数，如果运气好可以在 的时间复杂度下求解答案，但是对于 的数据，成功猜测的概率是，期望猜测的次数是。如果是在 里进行猜测，成功率会大一些。我们希望有方法来优化猜测。

朴素算法

最简单的算法即为从 进行遍历。



C++Python

```
1 vector<int> breakdown(int N) {
2     vector<int> result;
3     for (int i = 2; i * i <= N; i++) {
4         if (N % i == 0) { // 如果 i 能够整除 N, 说明 i 为 N 的一个质因子。
5             while (N % i == 0) N /= i;
6             result.push_back(i);
7         }
8     }
9     if (N != 1) { // 说明再经过操作之后 N 留下了一个素数
10        result.push_back(N);
11    }
12    return result;
13 }
```

```
1 def breakdown(N):
2     result = []
3     for i in range(2, int(sqrt(N)) + 1):
4         if N % i == 0: # 如果 i 能够整除 N, 说明 i 为 N 的一个质因子。
5             while N % i == 0:
6                 N //= i
7             result.append(i)
8     if N != 1: # 说明再经过操作之后 N 留下了一个素数
9         result.append(N)
10    return result
```

我们能够证明 `result` 中的所有元素即为 `N` 的全体素因数。

► 证明 `result` 中即为 的全体素因数

值得指出的是，如果开始已经打了一个素数表的话，时间复杂度将从 下降到 。去 [筛法](#) 处查阅更多打表的信息。

例题: [CF 1445C](#)

Pollard Rho 算法

引入

利用暴力算法获得一个非平凡因子的复杂度为 $O(\sqrt{n})$ ，这里， n 是 n 的最小素因子。而下面要介绍的 Pollard-Rho 算法是一种随机化算法，可以在 $O(n^{1/4})$ 的期望复杂度获得一个非平凡因子（注意！非平凡因子不一定是素因子）。

它的核心想法是，对于一个随机自映射，从任何一点出发，迭代计算，将在期望时间内进入循环。如果能够找到 x ，使得 $x^2 \equiv y \pmod{n}$ ，则 $x^2 - y$ 能被 n 整除，这一最大公约数就是 n 的一个非平凡因子。

要理解进入循环的期望时间为 $O(\sqrt{n})$ ，可以从生日悖论中获得启发。

生日悖论

不考虑出生年份（假设每年都是 365 天），问：一个房间中至少多少人，才能使其中两个人生日相同的概率达到 $\frac{1}{2}$ ？

解：假设一年有 d 天，房间中有 n 人，用整数 $1, 2, \dots, d$ 对这些人进行编号。假定每个人的生日均匀分布于 d 天之中，且两个人的生日相互独立。

设 n 个人生日互不相同为事件 A ，则事件 A 的概率为

至少有两个人生日相同的概率为 $1 - \frac{d!}{d^n}$ 。根据题意可知，那么就有

由不等式 $1 - x \leq e^{-x}$ 可得

因此

将 $x = \frac{n(n-1)}{2d}$ 代入，解得 $n \approx \sqrt{2d \ln 2}$ 。所以一个房间中至少 $\sqrt{2d \ln 2}$ 人，使其中两个人生日相同的概率达到 $\frac{1}{2}$ ，但这个数学事实十分反直觉，故称之为一个悖论。

当 $n = \sqrt{2d \ln 2}$ 时，出现两个人同一天生日的概率将大于 $\frac{1}{2}$ 。那么在一年有 d 天的情况下，当房间中有 n 个人时，至少有两个人的生日相同的概率约为 $\frac{1}{2}$ 。

类似地可以计算，随机均匀地选取一系列生日，首次获得重复生日需要的人数的期望也是 $\sqrt{\pi d}$ 。设这一人数为 n ，则

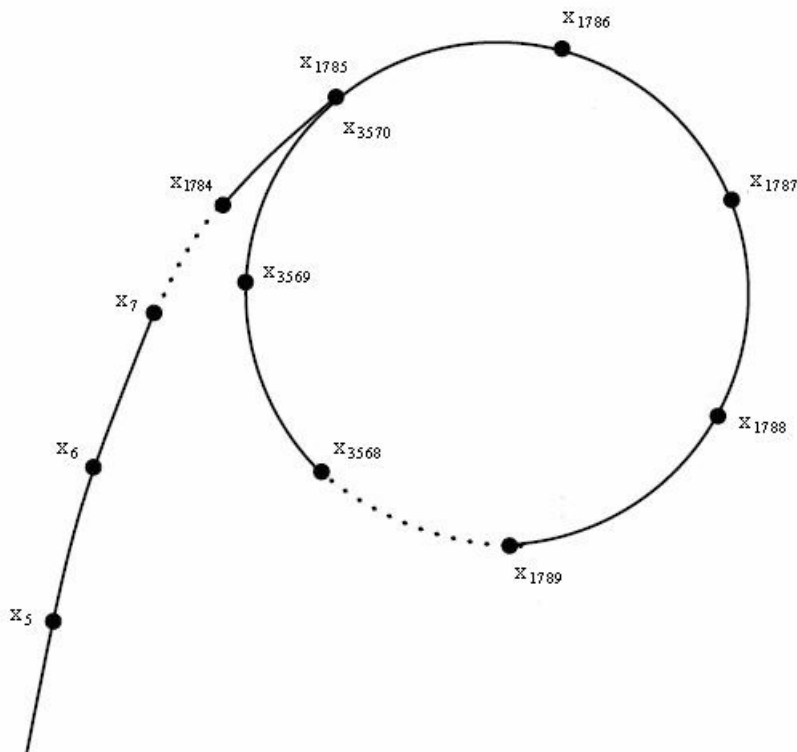
这启发我们，如果可以随机选取一系列数字，出现重复数字需要的抽样规模的期望也是 $\sqrt{\pi d}$ 。

利用最大公约数求出一个约数

实际构建一系列模 n 的随机数列并不现实，因为 n 正是要求的。所以，我们通过 $\text{rand}(a, b)$ 来生成一个伪随机数序列：随机取一个 x ，令 $y = (x^2 + c) \pmod{n}$ ，其中 c 是一个随机选取的常数。

这里选取的函数容易计算，且往往可以生成相当随机的序列。但它并不是完全随机的。举个例子，设 $n = 1000000007$ ，生成的数据为

可以发现数据在 0 以后都在 1000000000 之间循环。如果将这些数如下图一样排列起来，会发现这个图像酷似一个 ρ ，算法也因此得名 rho。



更重要的是，这样的函数确实提供了 f 上一个自映射。也就是说，它满足性质：如果 $x_i = x_j$ ，则 $x_{i+1} = x_{j+1}$ 。

▼ 证明

若 $x_i = x_j$ ，则 $x_{i+1} = x_{j+1}$ 。注意到， $x_{i+1} = f(x_i)$ ，这里 f 是一个依赖于 i 的整数，且 $f(i) = f(j)$ ，所以有 $x_{i+1} = x_{j+1}$ ，因而 $x_{i+2} = x_{j+2}$ 。

作为 f 上的伪随机自映射反复迭代得到的序列，在 n 的期望时间内就会出现重复。只要我们观察到这样的重复，就可以根据 $x_i = x_j$ 求出一个 n 的非平凡因子。注意到，由于 n 未知，我们并没有办法直接判断重复的发生，一个简单的判断方法正是 $\gcd(x_i - x_j, n) > 1$ 。

这一算法并不是总能成功的，因为 $\gcd(x_i - x_j, n)$ 可能等于 1。也就是说， $\gcd(x_i - x_j, n) = 1$ 。此时，首次发生重复时，恰好 $x_i = x_j$ 也发生重复了。我们没有得到一个非平凡因子。而且，开始循环后，再继续迭代也没有意义了，因为之后只会重复这一循环。该算法应输出分解失败，需要更换 n 中选取的 n 重新分解。

根据上文分析，理论上，任何满足 $f(x) = ax^2 + bx + c \pmod{n}$ ，且能够保证一定伪随机性的函数（例如某些多项式函数）都可以用在此处。实践中，主要使用 [2](#)。

实现

我们需要实现的算法，能够在迭代过程中快速判断 x_i 是否已经出现重复。将 f 看成以 x_i 为顶点的有向图上的边，我们实际要实现的是一个判环算法。只是将判等改为了判断 $\gcd(x_i - x_j, n) > 1$ 。

Floyd 判环

假设两个人在赛跑，A 的速度快，B 的速度慢，经过一定时间后，A 一定会和 B 相遇，且相遇时 A 跑过的总距离减去 B 跑过的总距离一定是圈长的倍数。

设 x_i ，每一次更新 x_i ，只要检查在更新过程中 x_i 和 x_j 是否相等，如果相等了，那么就出现了环。

我们每次令 $d = \gcd(x_i - x_j, n)$ ，判断 d 是否满足，若满足则可直接返回 d 。如果 $d = 1$ ，则说明 x_i 已经形成环，在形成环时就不能再继续操作了，直接返回 n 本身，并且在后续操作里调整随机常数，重新分解。

► 基于 Floyd 判环的 Pollard-Rho 算法

Brent 判环

实际上，Floyd 判环算法可以有常数上的改进。Brent 判环从 x 开始递增，在第 i 轮，让 A 等在原地， B 向前移动 i 步，如果在过程中 B 遇到了 A ，则说明已经得到环，否则让 A 瞬移到 B 的位置，然后继续下一轮。

可以证明³，这样得到环之前需要调用 next 的次数永远不大于 Floyd 判环算法。原论文中的测试表明，Brent 判环需要的平均时间相较于 Floyd 判环减少了 $\frac{1}{3}$ 。

倍增优化

无论是 Floyd 判环还是 Brent 判环，迭代次数都是 $O(\sqrt{n})$ 的。但是每次迭代都用 next 判断是否成环会拖慢算法运行速度。可以通过乘法累积来减少求 next 的次数。

简单来说，如果 $\text{next}^k(x) = x$ ，那么对于任意 k 都成立。也就是说，如果计算得到 $\text{next}^k(x)$ ，那么必然有其中一对 i, j 满足 $\text{next}^i(x) = \text{next}^j(x)$ 。如果该乘积在某一时刻得到零，则分解失败，退出并返回 x 本身。

如果每 k 对计算一次，则算法复杂度降低到 $O(\sqrt{n})$ ，这里 k 为单次计算的开销。注意到 k 和 $\sqrt{n}$ 大致同阶时，可以得到 $O(\sqrt{n})$ 的期望复杂度。具体实现中，大多选取 $k=2$ 。

这里提供 Brent 判环且加上倍增优化的 Pollard-Rho 算法实现。

► 实现

复杂度

Pollard-Rho 算法中的期望迭代次数为 $O(\sqrt{n})$ ，这里 n 是 n 的最小素因子。具体实现无论是采用 Floyd 判环还是 Brent 判环，如果不使用倍增优化，期望复杂度都是 $O(n)$ ；在加上倍增优化后，可以近似得到 $O(\sqrt{n})$ 的期望复杂度。

值得一提的是，前文分析基于的是完全随机的自映射函数，但 Pollard-Rho 算法实际使用的是伪随机函数，所以该算法并没有严格的复杂度分析，实践中通常跑得较快。

例题：求一个数的最大素因子

例题：P4718【模板】Pollard-Rho 算法

对于一个数 n ，用 Miller Rabin 算法判断是否为素数，如果是就可以直接返回了，否则用 Pollard-Rho 算法找一个因子，将 n 除去因子。再递归分解 n ，用 Miller Rabin 判断是否出现质因子，并用 max_factor 更新就可以求出最大质因子了。由于这个题目的数据过于庞大，用 Floyd 判环的方法是不够的，这里采用倍增优化的方法。

► 实现

参考资料与链接

1. https://en.wikipedia.org/wiki/Birthday_problem#Reverse_problem ↩
2. Menezes, Alfred J.; van Oorschot, Paul C.; Vanstone, Scott A. (2001). Handbook of Applied Cryptography. Section 3.11 and 3.12. ↩
3. Brent, R. P. (1980), An improved Monte Carlo factorization algorithm, BIT Numerical Mathematics, 20(2): 176–184, doi:10.1007/BF01933190 ↩

本页面贡献者：[iamtwz](#), [2740365712](#), [383494](#), [Backlight](#), [Bubbleioa](#), [c-forrest](#), [CCXXXI](#), [Enter-tainer](#), [GoodCoder666](#), [Great-designer](#), [lr1d](#), [kenlig](#), [leoleoasd](#), [megakite](#), [Menci](#), [PeterlitsZo](#), [Saisyc](#), [ShaoChenHeng](#), [shawleyw](#), [shuzhouliu](#), [StudyingFather](#), [TianKong-y](#), [Tiphereth-A](#), [usamoi](#), [Xeonacid](#), [xyf007](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用